Iterable and Iterator:

In programming, particularly in languages like Python, the concepts of iterables and iterators are fundamental for working with collections of data. Here's a detailed explanation of each:

Iterable:

An iterable is any object that can be looped over or iterated upon. Examples of iterables include ***lists, tuples, strings, dictionaries, sets, and more.***
Iterables must implement the ***__iter__()*** *method, which returns an iterator.*

```
fruitsList = ["apple", "banana", "cherry", "orange"]
print(dir(fruitsList))

for i in fruitsList:
    print(i)
```

```
['__add__', '__class__', '__class_getitem__', '__contains_
_', '__delattr__', '__delitem__', '__dir__', '__doc__', '_
_eq__', '__format__', '__ge__', '__getattribute__', '__get
item__', '__getstate__', '__gt__', '__hash__', '__iadd__',
'__imul__', '__init__', '__init_subclass__', '__iter__',
'__le__', '__len__', '__lt__', '__mul__', '__ne__', '__new
__', '__reduce__', '__reduce_ex__', '__repr__', '__reverse
d__', '__rmul__', '__setattr__', '__setitem__', '__sizeof_
_', '__str__', '__subclasshook__', 'append', 'clear', 'cop
y', 'count', 'extend', 'index', 'insert', 'pop', 'remove',
'reverse', 'sort']
apple
banana
cherry
orange
```

Python Iterators

In Python, an iterator is an object that implements the iterator protocol, which consists of two methods ***__iter__()*** *and* ***__next__()*** *.*

An example of Iterators are result of map and filter is iterators since both contain ***__iter__()*** *and* ***__next__()*** *.*

```
'''Apply Filter'''
fruitsList = [1,2,3,4,5,6,7,8]
def extractOnlyEvenNumber(i):
    return i % 2 == 0

abc = filter(extractOnlyEvenNumber,fruitsList)
print(abc)
print(dir(abc))

for i in abc:
    print(i)

```

```
<filter object at 0x00000248F8BAC0A0>
['__class__', '__delattr__', '__dir__', '__doc__', '__
eq__', '__format__', '__ge__', '__getattribute__', '__
getstate__', '__gt__', '__hash__', '__init__', '__init
_subclass__', '__iter__', '__le__', '__lt__', '__ne_
_', '__new__', '__next__', '__reduce__', '__reduce_ex_
_', '__repr__', '__setattr__', '__sizeof__', '__str_
_', '__subclasshook__']
2
4
6
8
```

Create an iterator from iterable

```
mystr = "banana"
myit = iter(mystr)
print(myit)
print(dir(myit))
print(next(myit))
print(next(myit))
print(next(myit))
print(next(myit))
print(next(myit))
print(next(myit))

```

```
<str_ascii_iterator object at 0x00000248F7F3AFB0>
['__class__', '__delattr__', '__dir__', '__doc__', '__eq__', '__format__', '__g
e__', '__getattribute__', '__getstate__', '__gt__', '__hash__', '__init__', '__
init_subclass__', '__iter__', '__le__', '__length_hint__', '__lt__', '__ne__',
'__new__', '__next__', '__reduce__', '__reduce_ex__', '__repr__', '__setattr_
_', '__setstate__', '__sizeof__', '__str__', '__subclasshook__']
b
a
n
a
n
a
```

```
fruituple=("apple", "banana", "cherry")
myit = iter(fruituple)
print(myit)
print(dir(myit))
print(next(myit))
print(next(myit))
print(next(myit))

```

```
<tuple_iterator object at 0x00000248F7ED97B0>
['__class__', '__delattr__', '__dir__', '__doc__', '__eq_
_', '__format__', '__ge__', '__getattribute__', '__getsta
te__', '__gt__', '__hash__', '__init__', '__init_subclass
__', '__iter__', '__le__', '__length_hint__', '__lt__',
'__ne__', '__new__', '__next__', '__reduce__', '__reduce_
ex__', '__repr__', '__setattr__', '__setstate__', '__size
of__', '__str__', '__subclasshook__']
apple
banana
cherry
```

Iterator vs Iterable:

Lists, tuples, dictionaries, and sets are all iterable objects. They are iterable containers which you can get an iterator from. All these objects have an iter() method which is used to get an iterator:

Iterable	***iterator***
An iterable is any object that can be looped over or iterated upon.	*An iterator is an object that represents a stream of data.*
Iterables must implement the ***__iter__()*** *method.*	*Iterators must implement the* ***__iter__()*** *and* ***__next__()*** *methods.*
Iterable objects can be passed to functions or constructs like ***for*** *loops directly.*	*The* ***__iter__()*** *method returns the iterator object itself.* *The* ***__next__()*** *method returns the next item in the stream. If there are no more items, it raises a StopIteration exception.*
Examples of iterables include lists, tuples, strings, dictionaries, sets, and more.	1) *An example is "result of map and filter" is iterators since both contain* ***__iter__()*** *and* ***__next__()*** *.* 2) *If we pass list, set,tupple in __iter__() it will return iterator.*

---

enumerates():

In Python, ***the enumerate()*** *function is used to iterate over a list (or any other iterable), providing both the* ***index*** *and the* ***value*** *of each element during iteration. This is particularly useful when you need to keep track of the index while iterating over the elements of a list.*

```
'''enumerate with List'''
fruitsList = ["apple", "banana", "cherry", "orange"]
for j, i in enumerate(fruitsList):
    print(j,i)
```

```
0 apple
1 banana
2 cherry
3 orange
```

```
'''enumerate with Tuple'''

fruitsTuple = ("apple", "banana", "cherry", "orange")
for j, i in enumerate(fruitsTuple):
    print(j,i)
```

```
0 apple
1 banana
2 cherry
3 orange
```